Multi-Repo Agent Orchestration with Layered Code Intelligence

Abstract

AI coding agents operating across multi-repository platforms lack cross-repo awareness, producing blind spots that lead to integration failures, compliance violations, and cascading breakage. This paper presents a three-layer intelligence hub composed behind a single Model Context Protocol (MCP) gateway that provides deterministic code-graph analysis, semantic memory retrieval, and live architecture-contract querying to AI agents working across a 13-repository fintech platform. On top of this intelligence layer sits a gated autonomous workflow with strict author-approver separation, tier-based safety escalation, grounding attestations, and honest enforcement accounting. The retrieval plane incorporates reranking, an evaluation harness, tool-usage observability, and a first-class governance corpus. Results demonstrate that layered composition eliminates the class of cross-repo integration failures that single-repo agents routinely produce in regulated environments.

1. Introduction

The adoption of AI coding agents in software engineering has accelerated rapidly, with tools like Claude Code, Cursor, and GitHub Copilot Workspace becoming standard in professional development workflows. These agents excel within a single repository: they read files, understand local context, write code, and run tests. However, production systems rarely live in a single repository.

In regulated fintech, the problem compounds. A platform handling tokenized real-world assets spans smart contracts, backend services, event processors, shared packages, and infrastructure configuration across dozens of repositories. A change to a shared compliance module ripples through every downstream consumer. An agent working in one service has zero awareness of upstream consumers, downstream breakage, or the architectural contracts that bind services together. Every cross-repo change becomes a coin flip.

This paper presents a system designed to solve this problem: a multi-repo agent orchestration hub that provides cross-repository intelligence and disciplined autonomous workflows for a 13-repository fintech platform built on cross-chain identity, compliance, and asset tokenization infrastructure.

2. Problem Statement

We identify four failure modes specific to AI agents operating across multi-repository platforms:

Blind blast radius. An agent modifying a shared package cannot determine which downstream repositories consume the changed exports. Without this knowledge, it cannot assess whether its changes will break consumers, trigger type errors, or violate API contracts.

Lost architectural context. Design decisions, compliance requirements, and historical rationale are distributed across repositories, issue trackers, and team knowledge. An agent operating in isolation makes decisions that conflict with established patterns or violate regulatory constraints it cannot see.

Stale system understanding. Agents grounded in training data operate on a frozen snapshot of the codebase. The live system diverges continuously. Routes are added, services are refactored, Kafka topics are renamed. An agent's understanding drifts from reality with every merge.

Ungated autonomy. Without structured gates, an autonomous agent can plan, implement, and ship changes that no independent reviewer has validated. In regulated fintech, where on-chain contract changes and compliance logic carry legal and financial consequences, ungated autonomy is unacceptable.

3. Architecture

The system composes three intelligence sub-servers behind a single FastMCP 3.x gateway, each mounted under a distinct namespace. An AI agent connects to one MCP endpoint and gains access to all three layers simultaneously.

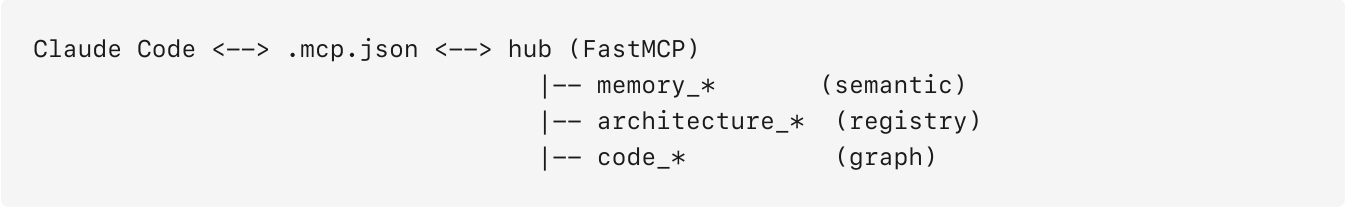

Figure 1. FastMCP composition gateway with three intelligence namespaces.

3.1 Code Graph Layer

The code graph provides deterministic, structural intelligence derived from AST-parsed property graphs. Source repositories are indexed by GitNexus, which extracts symbols, files, processes,

clusters, and edges into a property graph. The ingestion pipeline embeds searchable documents via Voyage Code 3 (with per-document truncation at 30K tokens) and upserts to per-repo Turbopuffer namespaces.

Key capabilities include:

- **Blast-radius analysis** (`code_impact`): BFS traversal over CALLS and IMPORTS edges with depth-bucketed risk scoring. Depth 1: WILL_BREAK. Depth 2: LIKELY_AFFECTED. Depth 3: MAY_NEED_TESTING.
- **360-degree symbol view** (`code_context`): Inbound and outbound edges with resolved neighbors for any symbol.
- **API surface diffing** (`code_shape_check`): Exported symbol comparison to detect breaking changes before they ship.
- **Route handler flow** (`code_route_map`): Per-step blast radius for API route changes.

Edge documents are stored as attribute-only records (no embeddings), saving approximately 26,000 Voyage API calls per re-ingestion cycle. Ingestion is triggered per-repo via GitHub Actions on push, with a daily scheduled run at 07:00 UTC. Staleness window is under two minutes post-merge.

3.2 Semantic Memory Layer

The semantic memory layer provides probabilistic intelligence: design rationale, precedent decisions, historical context, and accumulated knowledge across the platform.

The ingestion pipeline walks each repository via `git ls-files`, performs AST-aware chunking using Tree-sitter (TypeScript, TSX, Solidity) with fallback to text splitting. Target chunk sizes are 500–1,500 tokens. Chunks are embedded via Voyage Code 3 (1024-dimensional vectors) and upserted to Turbopuffer with content-hash deduplication.

Search uses hybrid retrieval: ANN (vector similarity) and BM25 (full-text) queries are executed via Turbopuffer's `multi_query` interface, then merged client-side using Reciprocal Rank Fusion (RRF, k=60) with configurable weighting (70% vector, 30% BM25). Caching is handled by Redis/Dragonfly with 7-day TTL for document embeddings and 5-minute TTL for search results.

3.3 Architecture Registry Layer

The architecture registry provides canonical, authoritative truth about the system's API surface. The `@signum-tech/contracts` package defines every route, service, process flow, and Kafka topic as

TypeScript code. The orchestrator loads this registry at runtime via `bun -e buildRegistry()` and exposes it as queryable MCP tools.

Tools include: `get_route` (with dependency chain, RBAC, and consuming services), `get_service` (with flow participation), `get_flow` (ordered steps with state transitions), `trace_topic` (producers, consumers, and flows), and `llms_txt` (canonical markdown rendering of the entire registry for AI prompt grounding).

3.4 Layer Composition

Each layer covers the others' blind spots. The code graph answers *what breaks* with deterministic precision. Semantic memory answers *why it was built that way* with probabilistic retrieval. The architecture registry answers *what the system promised* with canonical authority. Cross-cutting orchestrator tools compose across namespaces: `trace_package_impact` identifies all consuming repos for a changed package, `architecture_trace_flow` walks process flows and annotates each step with the implementing repo, and `search_workspace` fans out memory search across all repositories.

4. Retrieval Plane Enhancements

The base retrieval system is extended with four capabilities designed to close the feedback loop between retrieval quality and agent performance.

Reranking. Initial retrieval results from the hybrid ANN+BM25 pipeline are reranked using a cross-encoder model to improve precision for complex, multi-concept queries where lexical and semantic signals alone produce noisy results.

Evaluation harness. A retrieval evaluation framework measures recall, precision, and mean reciprocal rank against curated query-document pairs drawn from real agent sessions. This provides quantitative evidence of retrieval quality rather than relying on anecdotal agent performance.

Tool-usage observability. Every MCP tool invocation is instrumented with latency, token counts, cache hit rates, and result quality signals. A time-boxed Speakeasy Gateway pilot provides unified API observability across all MCP tool calls, enabling identification of underperforming tools and retrieval bottlenecks.

Governance corpus. A first-class governance corpus ingests security standards, internal policies, and company values into the RAG layer as a distinct document class. This ensures compliance

context is available at planning time, not just review time. When an agent plans a change that touches KYC flows or on-chain transfers, the governance corpus surfaces relevant policy constraints before implementation begins.

5. Gated Autonomous Workflow

The intelligence layer enables but does not govern autonomous work. Governance is provided by a structured, gated pipeline: `/ticket → /plan → /implement → /verify → /qa or /harden → /retro → /open-pr → STOP`.

5.1 Sub-Agent Context Isolation

Each phase runs as an isolated sub-agent with its own context window, returning a compact structured result via typed `*_COMPLETE` envelopes. The lifecycle orchestrator (`/flow`) holds only state transitions, two-sentence phase summaries, and gate decisions. It carries no `Edit` or `Write` tools. Every artifact is produced by a sub-agent it spawns. This prevents context window pollution and enforces separation of concerns.

5.2 Author-Approver Separation

The agent that produces an artifact never approves it. The planner does not bless its own spec. The implementer does not bless its own code. Independent verification and review passes ensure that every artifact is evaluated by a different agent than the one that created it.

5.3 Tier-Based Safety Escalation

Changes are classified into three tiers based on risk:

Tier	Scope	Review Required
A	On-chain contracts, RLS policies, KYC/AML logic, PII handling	9-pass fintech-grade harness
B	Medium-risk: API changes, auth flows, payment processing	9-pass harness
C	Standard: UI changes, documentation, tooling	Standard 2-pass review

Tier classification is re-evaluated after implementation against the actual diff, not just the plan. Code can drift into Tier A mid-flow if the implementation touches compliance-sensitive paths that weren't anticipated during planning.

5.4 Grounding Attestations

Every phase that relies on MCP tools records a grounding attestation: whether the tools answered, at what freshness, and whether the response was degraded. A degraded grounding is never presented as "verified" and never silently falls back to training memory. This creates an audit trail connecting AI-generated code to the live system state it was grounded in.

5.5 Honest Enforcement Accounting

The system explicitly categorizes each gate as either truly non-forgeable or discipline-only:

Gate	Enforcement
GitHub branch protection	Server-enforced, non-forgeable
CI pipeline checks	Server-enforced, non-forgeable
Human PR merge approval	Server-enforced, non-forgeable
In-session state.json gates	Discipline-only, auditable
Author-approver separation	Discipline-only, auditable
Tier reclassification	Discipline-only, auditable

This honest posture acknowledges that the LLM could forge a gate record. The system does not pretend otherwise. Discipline-only gates are explicitly planned for future mechanical enforcement via PreToolUse hooks.

5.6 Token Economics

LLM token costs are treated as a first-class architectural concern. Sub-agents are model-right-sized: Opus for planning (where reasoning quality is load-bearing), Sonnet for verification and review (where speed and cost matter more), Haiku for mechanical passes. Stable system prompts are placed first in the context to maximize prompt caching (~90% input discount). Tool surfaces are minimized per agent to reduce schema overhead.

6. The 9-Pass Review Harness

Tier A and B changes undergo a fintech-grade review protocol consisting of nine sequential passes:

1. **Evidence Ledger** — prove you read the code by citing file paths and line numbers for every factual claim.
2. **Scope Check** — verify only what was asked for was changed; nothing more.
3. **Comprehension Check** — explain the changed code line by line.
4. **Contract & Integration** — check for cross-repo breakage against the architecture registry.
5. **Failure Analysis** — enumerate failure modes and edge cases.
6. **Adversarial Analysis** — attempt to break the implementation.
7. **Compliance Verification** — validate against the governance corpus.
8. **Test Coverage Assessment** — verify test adequacy for the changed paths.
9. **Final Verdict** — PASS, CONDITIONAL, or BLOCK with specific remediation.

7. Autopilot Mode

For Tier C changes, the workflow supports an autopilot mode that removes in-flow human pauses. Three hard guardrails constrain autonomous execution:

- **G1:** Tier A/B reclassification on the actual diff halts autopilot immediately.
- **G2:** Execution always stops at PR-opened. No auto-merge, ever.
- **G3:** Degraded grounding or detected drift halts autopilot and drops to attended mode.

8. Discussion

Layer composition vs. monolithic retrieval. A single RAG pipeline over all repositories would conflate structural queries (what calls what?) with semantic queries (why was this designed this way?) and authoritative queries (what is the API contract?). The three-layer composition allows each query to be routed to the appropriate intelligence source. Deterministic questions get deterministic answers from the code graph. Probabilistic questions get ranked results from semantic memory. Canonical questions get authoritative answers from the architecture registry.

Honest enforcement as a design contribution. Most autonomous agent systems implicitly claim that their internal gates are enforced. This system's explicit distinction between server-enforced and discipline-only gates is itself load-bearing. It prevents false confidence in the system's safety properties and creates a clear roadmap for graduating discipline-only gates to mechanical enforcement.

Governance corpus at planning time. Traditional code review catches compliance violations after implementation. By surfacing governance constraints during planning, the system prevents compliance-violating designs from reaching implementation. This is particularly valuable in regulated fintech where architectural decisions have legal consequences.

9. Conclusion

This paper presents a multi-repo agent orchestration system that addresses the fundamental limitation of single-repo AI coding agents through layered intelligence composition. The three-layer MCP gateway provides deterministic, semantic, and authoritative intelligence to agents working across a 13-repository fintech platform. The gated autonomous workflow adds disciplined governance with honest enforcement accounting. The retrieval plane enhancements close the feedback loop between retrieval quality and agent performance, while the governance corpus ensures compliance context is available before implementation begins.

The system demonstrates that the gap between single-repo agent capability and multi-repo production requirements can be bridged through composition rather than monolithic expansion. Each intelligence layer, each workflow gate, and each retrieval enhancement addresses a specific failure mode observed in production agent usage. The result is an agent orchestration system where cross-repo awareness is not an afterthought but the foundational design constraint.

AK68A · ak68a.co · July 2026